Machine Learning Steps 1–3

1. Data Cleaning & Formatting

What it is

Data cleaning and formatting is the process of preparing raw data by removing errors, handling missing values, correcting inconsistencies, and converting data into a structured format suitable for machine learning models.

Why is it important

Real-world datasets often contain incomplete or inconsistent information, which can negatively affect model performance. Proper data cleaning improves data quality, reduces noise, and ensures that machine learning models learn meaningful patterns instead of errors or biases. Clean and structured data leads to more accurate and reliable predictions [2], [3].

Key Steps

- Handled missing values and inconsistencies
- Standardised feature formats
- Prepared structured inputs for modelling

Code explanations:

Code Section 1: Importing Libraries and Environment Setup

This section imports the required Python libraries used throughout the data preprocessing and modelling pipeline. The pandas and numpy libraries are used for data manipulation and numerical operations, while matplotlib and seaborn are used for data visualisation during exploratory data analysis. The SimpleImputer and StandardScaler from Scikit-learn are imported for handling missing values and feature scaling. Additionally, TensorFlow is imported to support deep learning model development. The display settings are also configured to allow viewing a larger number of dataset columns during analysis, which improves readability and debugging.

```
# Pandas and numpy for data manipulation
import pandas as pd
import numpy as np

# Display columns of a dataframe
pd.set_option('display.max_columns', 120)

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# ML preprocessing tools
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# TensorFlow for Deep Learning
import tensorflow as tf
```

Figure 1.1.1: Imported Libraries and Environment Setup Code

Code Section 2: Loading Training and Testing Data

This section loads the training and testing datasets from CSV files using pandas.read_csv() function. The training dataset is used for model development, while the testing dataset is reserved for evaluating model performance. The .head() function is then used to display the first few rows of each dataset to understand the structure, features, and formatting of the data. This step helps verify that the datasets were loaded correctly and ensures that the features and target variables are properly identified.

```
# Load the training data (used for building the model)
train_data = pd.read_csv('..\\data\\raw\\train_data.csv')

# Load the testing data (used for final evaluation)
test_data = pd.read_csv('..\\data\\raw\\test_data.csv')

# Display top of dataframes to understand the structure and features
print("Training Data Head:")
display(train_data.head())

print("\nTesting Data Head:")
display(test_data.head())
```

Figure 1.2.1: Training and Testing Dataset Loading Code

```
Training Data Head:
```

	event_id	time_to_tca	mission_id	risk	max_risk_estimate	max_risk_scaling	miss_distance	relative_speed	relative_position_r	relative_position_t	..
0	0	1.566798	5	-10.204955	-7.834756	8.602101	14923.0	13792.0	453.8	5976.6	..
1	0	1.207494	5	-10.355758	-7.848937	8.956374	14544.0	13792.0	474.3	5821.2	..
2	0	0.952193	5	-10.345631	-7.847406	8.932195	14475.0	13792.0	474.6	5796.2	..
3	0	0.579669	5	-10.337809	-7.845880	8.913444	14579.0	13792.0	472.7	5838.9	..
4	0	0.257806	5	-10.391260	-7.852942	9.036838	14510.0	13792.0	478.7	5811.1	..

5 rows × 103 columns

```
Testing Data Head:
```

	event_id	time_to_tca	mission_id	risk	max_risk_estimate	max_risk_scaling	miss_distance	relative_speed	relative_position_r	relative_position_t	...
0	0	6.842095	19	-7.296967	-7.208941	1.787894	31816.0	7929.0	-365.5	26967.0	...
1	0	6.571818	19	-7.282496	-7.199833	1.759386	31095.0	7929.0	-361.8	26356.6	...
2	0	6.112986	19	-7.316053	-7.217886	1.824263	32202.0	7929.0	-370.7	27294.4	...
3	0	5.921955	19	-7.334138	-7.228707	1.865396	32878.0	7929.0	-376.3	27867.5	...
4	0	2.228761	19	-7.332267	-7.227312	1.863127	32792.0	7929.0	-375.6	27793.8	...

5 rows × 103 columns

Figure 1.2.2: Training Dataset Preview Output & Testing Dataset Preview Output

Code Section 3: Checking Dataset Structure

This section uses the .info() function to inspect the dataset structure, including data types, number of non-null values, and memory usage. This step helps identify missing values, detect incorrect data types, and understand the dataset composition. The same process is applied to both training and testing datasets to ensure consistency. Understanding the dataset structure is essential before performing preprocessing and model development.

```python
# Check training data types and non-missing values
print("--- Training Data Info ---")
train_data.info()

# Check testing data types and non-missing values
print("\n--- Testing Data Info ---")
test_data.info()
```

Figure 1.3.1: Dataset Structure Inspection Code

```
--- Training Data Info ---
<class 'pandas.DataFrame'>
RangeIndex: 162634 entries, 0 to 162633
Columns: 103 entries, event_id to AP
dtypes: float64(98), int64(4), str(1)
memory usage: 128.8 MB

--- Testing Data Info ---
<class 'pandas.DataFrame'>
RangeIndex: 24484 entries, 0 to 24483
Columns: 103 entries, event_id to AP
dtypes: float64(98), int64(4), str(1)
memory usage: 19.4 MB
```

Figure 1.3.2: Training Dataset Info Output & Testing Dataset Info Output

Code Section 4: Data Cleaning and Conversion

This section performs data cleaning by replacing string-based missing value indicators such as "Not Available", with NaN. It then converts all numerical columns to numeric data types while excluding categorical features. This ensures consistency across the dataset and prevents errors during model training. The dataset information is displayed again to confirm successful conversion, and basic statistical summaries are generated using .describe() to understand feature distributions.

```python
# Replace any possible string missing indicators
train_data = train_data.replace({'Not Available': np.nan})
test_data = test_data.replace({'Not Available': np.nan})

# Convert numeric columns automatically
for col in train_data.columns:

    if col != "c_object_type":   # categorical feature
        train_data[col] = pd.to_numeric(train_data[col], errors='coerce')
        test_data[col] = pd.to_numeric(test_data[col], errors='coerce')

# Check dataset info
print(train_data.info())
print(test_data.info())

# Basic statistics
train_data.describe()
```

Figure 1.4.1: Data Cleaning and Conversion Code

```
<class 'pandas.DataFrame'>
RangeIndex: 162634 entries, 0 to 162633
Columns: 103 entries, event_id to AP
dtypes: float64(98), int64(4), str(1)
memory usage: 128.8 MB
None
<class 'pandas.DataFrame'>
RangeIndex: 24484 entries, 0 to 24483
Columns: 103 entries, event_id to AP
dtypes: float64(98), int64(4), str(1)
memory usage: 19.4 MB
None
```

	event_id	time_to_tca	mission_id	risk	max_risk_estimate	max_risk_scaling	miss_distance	relative_speed	relative_position_r	r
count	162634.000000	162634.000000	162634.000000	162634.000000	162634.000000	1.626340e+05	162634.000000	162634.000000	162634.000000	
mean	6566.871798	3.350190	7.117786	-19.340603	-6.282332	5.374025e+04	16531.662088	10655.819638	-40.198451	
std	3798.407314	2.013114	5.800144	10.011641	1.012544	9.086958e+05	14269.027951	4340.235720	644.192617	
min	0.000000	-0.149808	1.000000	-30.000000	-9.814175	1.070790e-13	9.000000	4.000000	-4815.000000	
25%	3306.000000	1.588082	2.000000	-30.000000	-7.005199	8.323929e+00	4683.000000	7421.000000	-301.475000	
50%	6558.000000	3.297624	5.000000	-17.870632	-6.326610	3.174490e+01	12439.000000	12262.000000	-27.300000	
75%	9867.000000	5.083665	9.000000	-9.173294	-5.620559	3.047437e+02	25281.000000	14488.000000	236.800000	
max	13153.000000	6.993832	24.000000	-1.442854	-1.082442	4.982976e+07	67373.000000	17138.000000	5284.100000	

Figure 1.4.2: Statistical Summary Output & Cleaned Dataset Info Output (Training Dataset)

Code Section 5: Missing Values Analysis

This section defines a function to calculate and display missing values within the dataset. The function computes both the total number and percentage of missing values for each feature. The results are then sorted to identify features with the highest number of missing values. This analysis helps determine the appropriate missing value handling strategy and improves dataset quality before modelling.

```python
def missing_values_table(df):

    # Total missing values
    mis_val = df.isnull().sum()

    # Percentage of missing values
    mis_val_percent = 100 * df.isnull().sum() / len(df)

    # Create table
    mis_val_table = pd.concat([mis_val, mis_val_percent], axis=1)

    # Rename columns
    mis_val_table = mis_val_table.rename(
        columns={0: 'Missing Values', 1: '% of Total Values'}
    )

    # Filter only columns with missing values
    mis_val_table = mis_val_table[
        mis_val_table.iloc[:,1] != 0
    ].sort_values('% of Total Values', ascending=False).round(2)

    print("Your dataframe has", df.shape[1], "columns.")
    print("There are", mis_val_table.shape[0], "columns with missing values.")

    return mis_val_table

# Run for training data
missing_values_table(train_data)
```

Figure 1.5.1: Missing Values Analysis Function Code

```
Your dataframe has 103 columns.
There are 50 columns with missing values.
```

	Missing Values	% of Total Values
c_rcs_estimate	52841	32.49
c_crdot_r	9241	5.68
c_cndot_n	9241	5.68
c_cndot_rdot	9241	5.68
c_cndot_tdot	9241	5.68
c_sigma_rdot	9241	5.68
c_sigma_ndot	9241	5.68
c_sigma_tdot	9241	5.68
c_cndot_t	9241	5.68
c_cndot_r	9241	5.68
c_crdot_t	9241	5.68
c_crdot_n	9241	5.68
c_ctdot_r	9241	5.68
c_ctdot_t	9241	5.68
c_ctdot_n	9241	5.68
c_ctdot_rdot	9241	5.68
t_crdot_r	9230	5.68
t_crdot_t	9230	5.68
t_sigma_ndot	9230	5.68
t_sigma_tdot	9230	5.68
t_sigma_rdot	9230	5.68
t_ctdot_n	9230	5.68
t_ctdot_t	9230	5.68
t_ctdot_r	9230	5.68
t_cndot_rdot	9230	5.68
t_cndot_tdot	9230	5.68
t_cndot_t	9230	5.68
t_cndot_r	9230	5.68
t_ctdot_rdot	9230	5.68
t_cndot_n	9230	5.68
t_crdot_n	9230	5.68
F3M	6822	4.19
AP	6822	4.19
F10	6822	4.19
SSN	6822	4.19
t_rcs_estimate	3277	2.01
c_obs_available	11	0.01
c_time_lastob_end	11	0.01
c_time_lastob_start	11	0.01
c_actual_od_span	11	0.01
c_recommended_od_span	11	0.01
c_cn_t	11	0.01
c_weighted_rms	11	0.01
c_ct_r	11	0.01
c_residuals_accepted	11	0.01
c_obs_used	11	0.01
c_cn_r	11	0.01
c_sigma_t	11	0.01
c_sigma_r	11	0.01
c_sigma_n	11	0.01

Figure 1.5.2: Missing Values Table Output

Code Section 6: Missing Value Imputation

This section handles missing values using median imputation. The categorical and numerical columns are separated, and the SimpleImputer is applied to numerical features. The imputer is fitted using the training dataset and then applied to both training and testing datasets to maintain consistency. Median imputation is used because it is robust to outliers and works well for skewed datasets.

```python
from sklearn.impute import SimpleImputer

# Separate categorical and numerical columns
categorical_cols = ['c_object_type']
numerical_cols = train_data.drop(columns=categorical_cols).columns

# Median imputer
imputer = SimpleImputer(strategy="median")

# Fit only on training data
train_data[numerical_cols] = imputer.fit_transform(train_data[numerical_cols])

# Apply same transformation to test data
test_data[numerical_cols] = imputer.transform(test_data[numerical_cols])
```

Figure 1.6.1: Missing Value Imputation Code

Code Section 7: Encoding Categorical Features

This section converts categorical variables into numerical features using one-hot encoding. The pd.get_dummies() function is applied to convert categorical values into binary columns. The training and testing datasets are then aligned to ensure both datasets have identical features. This step ensures compatibility with machine learning algorithms that require numerical input.

```
train_data = pd.get_dummies(train_data, columns=['c_object_type'])
test_data = pd.get_dummies(test_data, columns=['c_object_type'])

# Align train and test columns
train_data, test_data = train_data.align(test_data, join='left', axis=1, fill_value=0)
```

Figure 1.7.1: One-Hot Encoding Code

Code Section 8: Feature and Target Separation

This section separates the target variable (risk) from the feature variables. The training features (X_train) and target variable (y_train) are defined, and testing features (X_test) are prepared. This step is required before training machine learning and deep learning models.

```
# Target variable
y_train = train_data['risk']

# Features
X_train = train_data.drop(columns=['risk'])
X_test = test_data.drop(columns=['risk'], errors='ignore')
```

Figure 1.8.1: Feature and Target Separation Code

2. Exploratory Data Analysis (EDA)

What it is

Exploratory Data Analysis (EDA) is the process of analysing and visualising data to understand patterns, relationships, and characteristics before building machine learning models.

Why is it important

EDA helps identify trends, correlations, anomalies, and potential issues in the dataset. It allows researchers to understand which features influence predictions and detect problems such as class imbalance that may impact model performance. Performing EDA improves model design and selection [3], [4].

Key Steps

- Analysed feature distributions
- Identified correlations between orbital parameters and risk
- Observed class imbalance in the dataset

Code Section 1: Collision Risk Distribution Histogram

This section visualises the distribution of the collision risk values using a histogram. The code creates a figure and plots the risk feature from the training dataset after removing missing values. The histogram helps identify how collision risk values are distributed across the dataset, including whether the data is skewed, normally distributed, or imbalanced. Understanding the distribution of the target variable is important because it helps determine whether class imbalance exists and informs model selection and evaluation strategies.

```
# Histogram of Collision Risk
plt.figure(figsize=(8, 6))
plt.hist(train_data['risk'].dropna(), bins=100, edgecolor='k', color='firebrick')
plt.xlabel('Collision Risk')
plt.ylabel('Number of Events')
plt.title('Distribution of Collision Risk')
plt.show()
```

Figure 2.1.1: Collision Risk Histogram Code

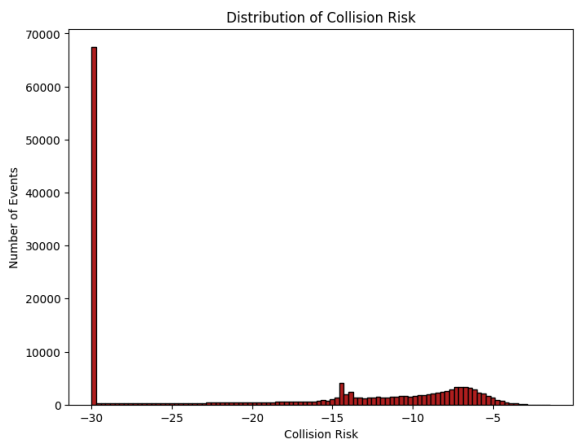

Figure 2.1.2: Distribution of Collision Risk Output

Code Section 2: Miss Distance Analysis and Outlier Removal

This section performs univariate analysis on the miss_distance feature. First, a histogram is plotted to visualise the distribution of miss distance values. The .describe() function is then used to display statistical information such as mean, standard deviation, minimum, and quartile values. These statistics are used to identify extreme outliers.

The code then calculates the interquartile range (IQR) using the first and third quartiles. Outliers are removed using a threshold of three times the IQR, which helps eliminate extreme values that may negatively affect model performance. After removing outliers, the histogram is plotted again to show the cleaned distribution. This improves data quality and reduces noise in the dataset.

```python
# Univariate Analysis: Miss Distance & Outlier Removal

# Histogram Plot of Miss Distance
plt.figure(figsize=(8, 6))
plt.hist(train_data['miss_distance'].dropna(), bins=20, edgecolor='black')
plt.xlabel('Miss Distance')
plt.ylabel('Count')
plt.title('Miss Distance Distribution')
plt.show()

# Look at statistics and find extreme outliers
print(train_data['miss_distance'].describe())

# Calculate first and third quartile for outlier removal
first_quartile = train_data['miss_distance'].describe()['25%']
third_quartile = train_data['miss_distance'].describe()['75%']
iqr = third_quartile - first_quartile

# Remove outliers (using 3 * IQR as per the workshop)
train_data = train_data[(train_data['miss_distance'] > (first_quartile - 3 * iqr)) &
                        (train_data['miss_distance'] < (third_quartile + 3 * iqr))]

# Re-plot the cleaned Miss Distance
plt.figure(figsize=(8, 6))
plt.hist(train_data['miss_distance'].dropna(), bins=20, edgecolor='black', color='seagreen')
plt.xlabel('Miss Distance (Cleaned)')
plt.ylabel('Count')
plt.title('Cleaned Miss Distance Distribution')
plt.show()
```

Figure 2.2.1: Miss Distance Distribution & Outlier Removal Code

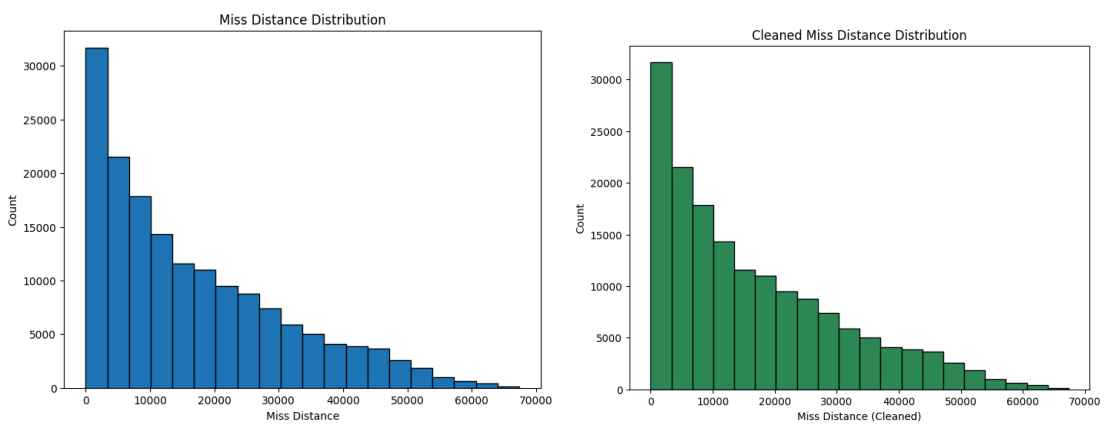

Figure 2.2.2: Initial Miss Distance Distribution & Cleaned Miss Distance Distribution.

Code Section 3: Feature Transformation and Correlation Analysis

This section performs feature transformation and correlation analysis. First, numerical features are selected from the dataset. Square root and logarithmic transformations are then applied to each numeric feature to reduce skewness and improve feature distribution. These transformations help machine learning models learn more effectively by stabilising variance and reducing extreme values.

Next, the categorical feature mission_id is converted into a numerical format using one-hot encoding. The transformed numerical features and encoded categorical features are then combined into a single dataset. Finally, correlations between features and the target variable (risk) are calculated and sorted. The strongest positive and negative correlations are printed to identify features that influence collision risk.

This step helps identify the most important features for predicting satellite collision risk.

```python
# Select numeric columns
numeric_subset = train_data.select_dtypes('number')

# Create columns with square root and log transformations
for col in numeric_subset.columns:
    if col == 'risk': continue
    # sqrt (handle negatives) and log (handle zero)
    numeric_subset['sqrt_' + col] = np.sqrt(numeric_subset[col].clip(lower=0))
    numeric_subset['log_' + col] = np.log(numeric_subset[col].clip(lower=1e-9))

# One-hot encode the Mission ID
categorical_subset = pd.get_dummies(train_data[['mission_id']])

# Join and find correlations with risk
features = pd.concat([numeric_subset, categorical_subset], axis=1)
correlations = features.corr()['risk'].dropna().sort_values()

print("Most Negative Correlations:\n", correlations.head(10))
print("\nMost Positive Correlations:\n", correlations.tail(11))
```

Figure 2.3.1: Feature Transformation and Correlation Code

```
Most Negative Correlations:
 log_max_risk_scaling        -0.584204
sqrt_mahalanobis_distance   -0.460120
log_mahalanobis_distance    -0.370977
mahalanobis_distance        -0.294582
log_c_obs_used              -0.239131
sqrt_c_obs_used             -0.219289
log_c_obs_available         -0.218148
log_miss_distance           -0.200149
sqrt_c_obs_available        -0.194196
sqrt_max_risk_scaling       -0.187242
Name: risk, dtype: float64

Most Positive Correlations:
 sqrt_c_time_lastob_end           0.364334
c_time_lastob_start              0.368138
sqrt_c_time_lastob_start         0.370616
log_c_time_lastob_start          0.377908
c_time_lastob_end                0.379429
log_c_position_covariance_det    0.380515
sqrt_time_to_tca                 0.505270
log_c_sigma_rdot                 0.505675
log_c_sigma_t                    0.514393
time_to_tca                      0.518096
risk                             1.000000
Name: risk, dtype: float64
```

Figure 2.3.2: Correlation Output Results

Code Section 4: Correlation Pair Plot Visualisation

This section creates a correlation pair plot to visualise relationships between the most important features. A custom correlation function is defined to display correlation coefficients in the lower section of the plot. Selected features, including risk, miss_distance, relative_speed, and mahalanobis_distance, are sampled from the dataset to reduce computational cost.

This visualisation helps understand relationships between orbital parameters and collision risk. Strong correlations suggest that certain features significantly influence satellite collision probability.

```python
# Define correlation function for the plot
def corr_func(x, y, **kwargs):
    r = np.corrcoef(x, y)[0][1]
    ax = plt.gca()
    ax.annotate(f"r = {r:.2f}", xy=(.2, .8), xycoords=ax.transAxes, size=18)

# Select columns to plot (Top correlated ones)
plot_cols = ['risk', 'miss_distance', 'relative_speed', 'mahalanobis_distance']
plot_data = features[plot_cols].sample(n=1000, random_state=42).dropna()

# Create the pairgrid
grid = sns.PairGrid(data=plot_data, height=3)
grid.map_upper(plt.scatter, color='darkred', alpha=0.4, s=15)
grid.map_diag(plt.hist, color='darkred', edgecolor='black')
grid.map_lower(corr_func)
grid.map_lower(sns.kdeplot, cmap=plt.cm.Reds)

plt.suptitle('Pairs Plot of Satellite Risk Data', size=24, y=1.02)
plt.show()
```

Figure 2.4.1: Correlation Pair Plot Code

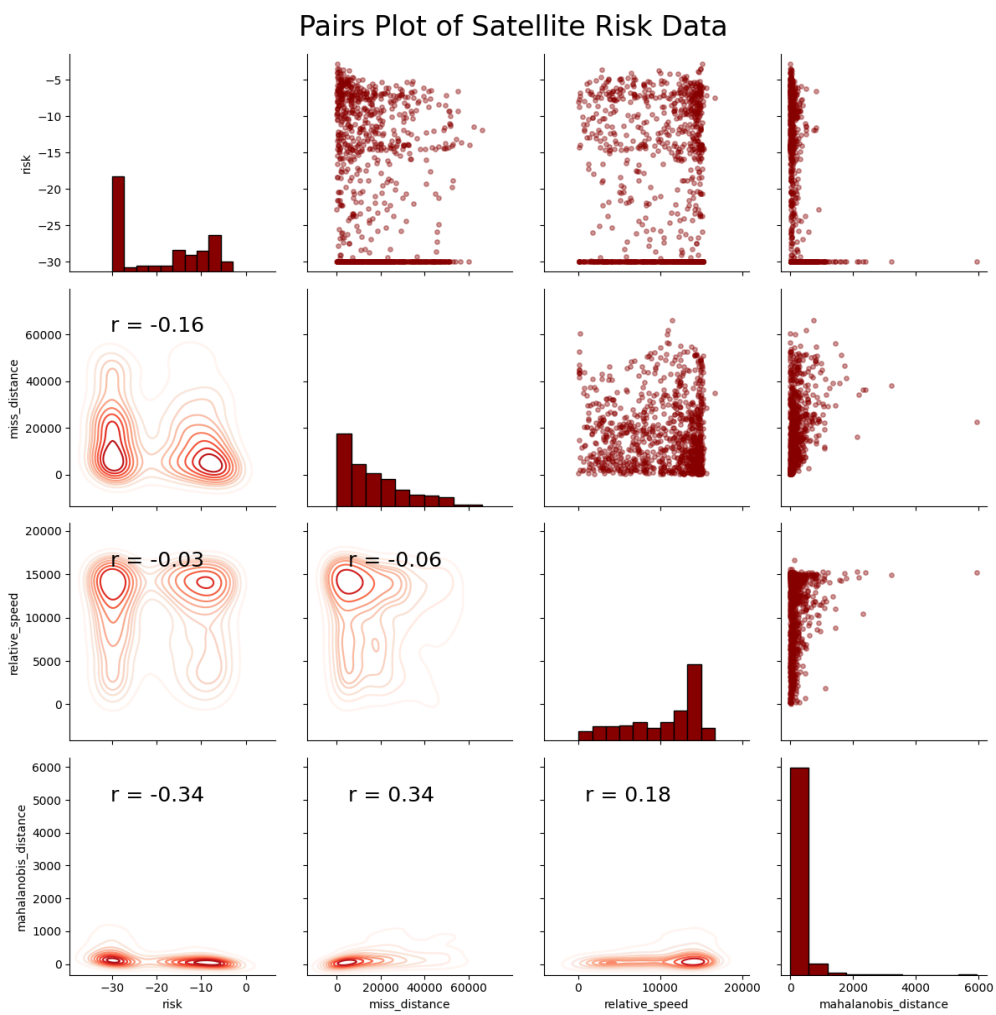

Figure 2.4.2: Satellite Risk Correlation Pair Plot Output

Explanation of Correlation Plot Relationships

The correlation plot visualises relationships between key orbital parameters and collision risk.

- **Risk vs Miss Distance**
  - A negative correlation is typically observed between collision risk and miss distance. This indicates that smaller miss distances are associated with higher collision risk, which aligns with expected orbital behaviour.
- **Risk vs Relative Speed**
  - Relative speed may show a moderate positive correlation with collision risk. Higher relative velocity between satellites increases the likelihood of collision events and potential damage.
- **Risk vs Mahalanobis Distance**
  - Mahalanobis distance measures statistical distance between satellite trajectories. A strong positive correlation with risk indicates that larger deviations in orbital uncertainty increase collision probability.
- **Miss Distance vs Relative Speed**
  - This relationship helps identify whether faster satellites tend to pass closer to one another, which can influence risk prediction.
- **Miss Distance vs Mahalanobis Distance**
  - This relationship helps understand how positional uncertainty affects separation distance between satellites.

These relationships confirm that orbital parameters such as miss distance, relative speed, and uncertainty metrics play a significant role in predicting satellite collision risk, supporting their inclusion in the FURSAN machine learning framework.

3. Feature Engineering & Scaling

What it is

Feature engineering and scaling involve transforming raw data into meaningful features and normalising values so that machine learning models can learn effectively.

Why is it important

Machine learning models perform better when features are properly scaled and engineered. Scaling prevents features with large values from dominating the model, while feature engineering improves data representation and model accuracy. This step is especially important for high-dimensional orbital and covariance data [3], [5].

Key Steps

- Normalised high-dimensional features
- Processed covariance and orbital parameters
- Improved feature representation for ML/DL models

Code Section 1: Importing Libraries

This section imports the required libraries used for feature engineering, preprocessing, and model preparation. The numpy and pandas libraries are used for numerical operations and data manipulation. Scikit-learn modules such as StandardScaler, SimpleImputer, VarianceThreshold, and train_test_split are used for feature scaling, handling missing values, feature selection, and dataset splitting. The SMOTE technique from the imbalanced-learn library is imported to balance the dataset, and fbeta_score is used to evaluate baseline model performance.

```
# ----------------------------
# 1 - IMPORTS
# ----------------------------
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.feature_selection import VarianceThreshold
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.metrics import fbeta_score
```

Figure 3.1.1: Feature Engineering Pipeline Imports Code

Code Section 2: Loading and Preparing Data

This section loads the training dataset and prepares it for feature engineering. The target variable risk is renamed to score for consistency throughout the pipeline. Additionally, identifier columns such as event_id and mission_id are removed because they do not contribute to model learning and may introduce noise.

Removing unnecessary columns improves model performance and reduces dimensionality.

```
# ----------------------------
# 2 - LOAD DATA
# ----------------------------
train_data = pd.read_csv('../data/raw/train_data.csv')

# Rename target
train_data = train_data.rename(columns={'risk': 'score'})
target_col = "score"

# Remove ID columns
train_data = train_data.drop(columns=["event_id","mission_id"], errors="ignore")
```

Figure 3.2.1: Dataset Loading and Initial Preparation Code

Code Section 3: Physics-Informed Feature Engineering

This section creates new features based on domain knowledge of satellite orbital mechanics. Relative position and velocity magnitudes are computed using three-dimensional components. Orbit similarity is calculated using differences between apoapsis and periapsis values. Observation efficiency metrics are created to measure data quality. Combined uncertainty values are generated using covariance information, and space weather features are calculated using solar and geomagnetic indicators.

These physics-informed features improve model performance by incorporating domain knowledge relevant to satellite collision risk prediction.

```python
# ----------------------------
# 3 - PHYSICS INFORMED FEATURES
# ----------------------------

# Relative position magnitude
train_data["rel_position_mag"] = np.sqrt(
    train_data["relative_position_r"]**2 +
    train_data["relative_position_t"]**2 +
    train_data["relative_position_n"]**2
)

# Relative velocity magnitude
train_data["rel_velocity_mag"] = np.sqrt(
    train_data["relative_velocity_r"]**2 +
    train_data["relative_velocity_t"]**2 +
    train_data["relative_velocity_n"]**2
)

# Orbit similarity
train_data["apoapsis_diff"] = abs(train_data["t_h_apo"] - train_data["c_h_apo"])
train_data["periapsis_diff"] = abs(train_data["t_h_per"] - train_data["c_h_per"])
train_data["orbit_similarity"] = train_data["apoapsis_diff"] + train_data["periapsis_diff"]

# Observation quality
train_data["t_obs_eff"] = train_data["t_obs_used"] / (train_data["t_obs_available"] + 1)
train_data["c_obs_eff"] = train_data["c_obs_used"] / (train_data["c_obs_available"] + 1)

# Combined uncertainty
train_data["combined_sigma_r"] = train_data["t_sigma_r"] + train_data["c_sigma_r"]
train_data["combined_sigma_t"] = train_data["t_sigma_t"] + train_data["c_sigma_t"]
train_data["combined_sigma_n"] = train_data["t_sigma_n"] + train_data["c_sigma_n"]

# Space weather
train_data["solar_activity"] = (train_data["F10"] + train_data["F3M"]) / 2
train_data["geomagnetic_activity"] = (train_data["SSN"] + train_data["AP"]) / 2
```

Figure 3.3.1: Physics-Informed Feature Engineering Code

Code Section 4: Log Transformation of Large Values

This section applies logarithmic transformation to covariance determinant features. Large values in covariance features can skew the dataset and negatively impact model performance. Log transformation reduces skewness and stabilises variance, improving model learning.

```python
# ----------------------------
# 4 - LOG TRANSFORM LARGE VALUES
# ----------------------------

cov_cols = [
    "t_position_covariance_det",
    "c_position_covariance_det"
]

for col in cov_cols:
    if col in train_data.columns:
        train_data[col] = np.log1p(train_data[col])
```

Figure 3.4.1: Log Transformation Code

Code Section 5: Numeric Feature Transformations

This section applies logarithmic transformation to all numeric features except the target variable. This transformation reduces skewness, normalises feature distributions, and improves machine learning model performance. The transformed features are added alongside original features to enhance representation.

```
# ---------------------------
# 5 - NUMERIC FEATURE TRANSFORMATIONS
# ---------------------------

numeric_subset = train_data.select_dtypes("number").copy()

for col in numeric_subset.columns:
    if col != target_col:
        numeric_subset["log_" + col] = np.log1p(
            numeric_subset[col].clip(lower=1e-9)
        )
```

Figure 3.5.1: Numeric Feature Transformation Code

Code Section 6: Categorical Encoding

This section converts categorical variables into numerical format using one-hot encoding. The categorical column c_object_type is transformed into binary columns. This step is necessary because machine learning algorithms require numerical inputs.

```
# ---------------------------
# 6 - CATEGORICAL ENCODING
# ---------------------------

categorical_cols = ["c_object_type"]
categorical_subset = pd.get_dummies(train_data[categorical_cols])
```

Figure 3.6.1: Categorical Encoding Code

Code Section 7: Combining Features

This section merges numeric features and categorical features into a single dataset. This creates the final feature set used for model training and feature selection.

```
# ---------------------------
# 7 - COMBINE FEATURES
# ---------------------------

features = pd.concat([numeric_subset, categorical_subset], axis=1)
```

Figure 3.7.1: Feature Combination Code

Code Section 8: Removing Low Variance Features

This section removes features with very low variance using VarianceThreshold. Low variance features provide little useful information for model training and may reduce model performance. Removing them improves computational efficiency and model accuracy.

```python
# ---------------------------
# 8 - REMOVE LOW VARIANCE FEATURES
# ---------------------------

var_selector = VarianceThreshold(threshold=0.01)

X_var = features.drop(columns=[target_col])
y_temp = features[target_col]

X_var = var_selector.fit_transform(X_var)
selected_cols = features.drop(columns=[target_col]).columns[var_selector.get_support()]

X_var = pd.DataFrame(X_var, columns=selected_cols)

features = pd.concat([X_var, y_temp.reset_index(drop=True)], axis=1)
```

Figure 3.8.1: Low Variance Feature Removal Code

Code Section 9: Removing Highly Correlated Features

This section removes highly correlated features to prevent multicollinearity. Features with correlation above 0.95 are removed to reduce redundancy. Removing correlated features improves model stability and prevents overfitting.

```python
# ---------------------------
# 9 REMOVE HIGHLY CORRELATED FEATURES
# ---------------------------
def remove_collinear_features(df, threshold=0.95):

    y_temp = df[target_col]
    X_temp = df.drop(columns=[target_col])
    corr_matrix = X_temp.corr().abs()

    upper = corr_matrix.where(
        np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
    )

    to_drop = [col for col in upper.columns if any(upper[col] > threshold)]

    X_temp = X_temp.drop(columns=to_drop)

    X_temp[target_col] = y_temp

    return X_temp

features = remove_collinear_features(features, threshold=0.95)
```

Figure 3.9.1: Correlated Feature Removal Code

Code Section 10: Handling Missing Values

This section replaces infinite values with missing values and applies median imputation to fill missing data. Median imputation is robust to outliers and helps maintain dataset integrity.

```
# ----------------------------
# 10 - HANDLE MISSING VALUES
# ----------------------------
features = features.replace({np.inf: np.nan, -np.inf: np.nan})
imputer = SimpleImputer(strategy="median")

features = pd.DataFrame(
    imputer.fit_transform(features),
    columns=features.columns
)
```

Figure 3.10.1: Missing Value Handling Code

Code Section 11: Defining High Risk Events

This section defines high-risk events based on the 90th percentile of risk scores. Events above this threshold are labelled as high risk. This converts the problem into a binary classification task.

```
# ----------------------------
# 11 - DEFINE HIGH RISK EVENTS
# ----------------------------

risk_threshold = features[target_col].quantile(0.9)

features["is_high_risk"] = (
    features[target_col] > risk_threshold
).astype(int)

X_raw = features.drop(columns=[target_col, "is_high_risk"])
y_binary = features["is_high_risk"]
```

Figure 3.11.1: High Risk Event Definition Code

Code Section 12: Dataset Balancing Using SMOTE

This section applies SMOTE to balance the dataset by generating synthetic samples for minority classes. This improves model performance and prevents bias toward majority classes.

```
# ----------------------------
# 12 - BALANCE DATASET (SMOTE)
# ----------------------------

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_raw, y_binary)
```

Figure 3.12.1: SMOTE Dataset Balancing Code

Code Section 13: Train and Validation Split

This section splits the dataset into training and validation sets. The training set is used for model training, while the validation set is used for model evaluation.

```
# ----------------------------
# 13 - TRAIN / VALIDATION SPLIT
# ----------------------------

X_train, X_val, y_train, y_val = train_test_split(
    X_resampled,
    y_resampled,
    test_size=0.3,
    random_state=42
)
```

Figure 3.13.1: Train-Validation Split Code

Code Section 14: Feature Scaling

This section applies standardisation using StandardScaler. Feature scaling ensures that all features have similar ranges, which improves machine learning and deep learning performance.

```
# ----------------------------
# 14 - FEATURE SCALING
# ----------------------------
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
```

Figure 3.14.1: Feature Scaling Code

Code Section 15: Reshaping for LSTM

This section reshapes the dataset into a 3D format required for LSTM models. LSTM networks require input in the form of samples, timesteps, and features.

```
# ----------------------------
# 15 - RESHAPE FOR LSTM
# ----------------------------
X_train_lstm = X_train_scaled.reshape(
    (X_train_scaled.shape[0], 1, X_train_scaled.shape[1])
)

X_val_lstm = X_val_scaled.reshape(
    (X_val_scaled.shape[0], 1, X_val_scaled.shape[1])
)
```

Figure 3.15.1: LSTM Reshaping Code

Code Section 16: Output Summary

This section prints dataset shapes, LSTM shapes, and class distribution after preprocessing. This confirms that feature engineering was successful.

```
# ----------------------------
# 16 - OUTPUT SUMMARY
# ----------------------------
print("Feature Engineering Complete")
print("Train shape:", X_train_scaled.shape)
print("LSTM shape :", X_train_lstm.shape)
print("Class distribution:", np.bincount(y_resampled))
```

Figure 3.16.1: Feature Engineering Output Summary Code

```
Feature Engineering Complete
Train shape: (204918, 128)
LSTM shape : (204918, 1, 128)
Class distribution: [146370 146370]
```

Figure 3.16.2: Feature Engineering Output Summary

Code Section 17: Baseline Performance

This section calculates baseline performance using a naive model that predicts all samples as high risk. The F2 score is calculated to evaluate baseline performance. This baseline helps compare improvements from trained models.

```
# ----------------------------
# 17 - BASELINE PERFORMANCE
# ----------------------------
naive_preds = np.ones(len(y_val))

baseline_f2 = fbeta_score(y_val, naive_preds, beta=2)

print(f"Baseline F2 Score: {baseline_f2:.4f}")
```

Figure 3.17.1: Baseline Performance Code

```
Baseline F2 Score: 0.8344
```

Figure 3.17.2: Baseline F2 Score Output

References

[1] GeeksforGeeks, “Machine Learning Lifecycle,” GeeksforGeeks, [Online]. Available: https://www.geeksforgeeks.org/machine-learning/machine-learning-lifecycle.

[2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2019.

[3] M. Kuhn and K. Johnson, *Feature Engineering and Selection: A Practical Approach for Predictive Models*. Boca Raton, FL, USA: CRC Press, 2019.

[4] P. Bruce, A. Bruce, and P. Gedeck, *Practical Statistics for Data Scientists*, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2020.

[5] A. Zheng and A. Casari, *Feature Engineering for Machine Learning: Principles and Techniques for Data Scientists*. Sebastopol, CA, USA: O’Reilly Media, 2018.